AI-Based Automated Multi-Organ Segmentation in Medical Imaging using UNet-ResNet34

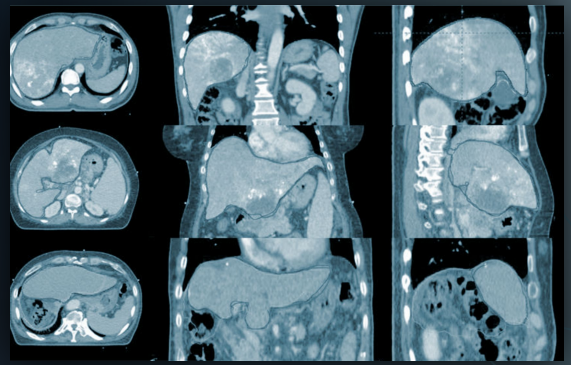
Niyati Nikunj Kapadia
Machine Vision-CECS 553 Sec01
November 15, 2025

Agenda

1. Introduction & Motivation
2. Case Study: The Need for Automated Organ Segmentation
3. Dataset Construction & Fusion
4. Dataset Verification
5. Patch Extraction
6. Training Pipeline
7. Model Evaluation
8. Test Dataset & Single Image Prediction
9. Challenges Faced
10. Conclusion
11. Future Work
12. References

Introduction & Motivation

- Medical imaging plays a vital role in diagnosis and treatment.
- Manual organ segmentation is slow and inconsistent.
- Increasing patient scans → higher workload for radiologists.
- Deep learning offers a faster, reproducible solution.
- Aim: Build an automated, efficient, and accurate multi-organ segmentation model.



Case Study: Why Automation is Needed

- Manual segmentation: ~15–60 minutes per CT scan, depending on complexity. (academic.oup.com, 2023)
- Imaging workload: ↑ ~300% over last 15 years for on-call CT. (healthimaging.com, 2022)
- Inter-observer variation: 15–25% in manual organ segmentation. (pubmed.ncbi.nlm.nih.gov, 2022)
- Pancreatic cancer survival rate: <10%. (WHO, 2024)
- AI-based segmentation: up to 90% faster than manual methods, with consistent accuracy. (academic.oup.com , 2023
- Need: Automated, consistent, and scalable segmentation system to reduce workload and improve reproducibility.

Dataset Construction & Fusion

Combined Dataset Builder

- Merge Pancreas and LiTS datasets into a 5-class segmentation dataset:
- 0: background, 1: liver, 2: liver tumor, 3: pancreas, 4: pancreas tumor
- Pancreas: align images with organ and tumor masks
- LiTS: regex + key-based pairing to handle inconsistent file names
- Export: combined__000000.png, all organized in unified directory

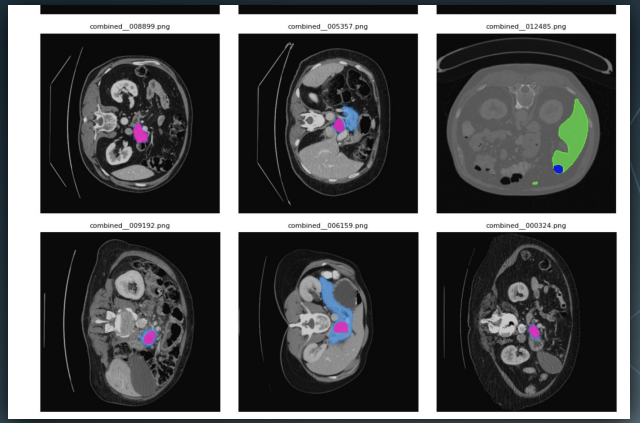
Packaging

- Dataset archived as /kaggle/working/combined_dataset.zip
- Ready for training workflows

Dataset Verification

Mask Value Verification

- Scan masks to ensure presence of all 5 class labels (0–4)
- Prevents encoding errors or missing classes



Combined Dataset Inspection

- Class balance analysis (pixel count per class)
- Visual inspection using vivid color overlays

Outcome

- Dataset is consistent, balanced, and visually verified

Patch Extraction

- Sliding window patches (PATCH_SIZE=128, PATCH_STRIDE=64) extracted from images.
- Skipped patches with only background.
- Recorded dominant class per patch for class-balanced sampling.
- Saved patches to /kaggle/working/patches.

Benefits

- Balances classes in training dataset.
- Makes training memory-efficient.
- Small patches focus on organ/tumor regions → better model learning.

Training Pipeline

UNet + ResNet34 Backbone

- Mixed-precision training
- Channels-last memory format

Two-stage training:

- Encoder frozen
- Unfreeze last ResNet blocks for fine-tuning
- Dice + CrossEntropy hybrid loss
- Balanced batch sampling

Pipeline Steps

- Reproducible environment setup
- Data loading & augmentations
- Model definition & two-stage optimization
- Class-balanced WeightedRandomSampler based on dominant class per patch.
- Training loop with checkpointing & early stopping
- Final Training Result: Epoch 10/10 | Train Loss: 0.0100 | Val Dice: 0.9676 | 343.5s

Training Pipeline

```python
class PatchDataset(Dataset):
    def __init__(self, img_dir, msk_dir, transform=None, cache_dom=DOM_CACHE):
        self.img_dir = img_dir
        self.msk_dir = msk_dir
        self.files = sorted(os.listdir(img_dir))
        self.transform = transform

        # load or compute dominant classes
        if os.path.exists(cache_dom):
            self.patches = np.load(cache_dom, allow_pickle=True).tolist()
            if len(self.patches) != len(self.files):
                self.patches = None
        else:
            self.patches = None

        if self.patches is None:
            print("Scanning masks — running once...")
            dom = []
            for idx, fname in enumerate(tqdm(self.files)):
                m = cv2.imread(os.path.join(msk_dir, fname), 0)
                if m is None:
                    m = np.zeros((128,128), np.uint8)
                m = np.clip(m, 0, 4)
                dominant = int(np.bincount(m.flatten(), minlength=len(CLASSES)).argmax())
                dom.append((idx, dominant))
            self.patches = dom
            np.save(cache_dom, self.patches, allow_pickle=True)

    def __len__(self):
        return len(self.files)
```

```python
class DoubleConv(nn.Module):
    def __init__(self, in_c, out_c):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(in_c, out_c, 3, padding=1, bias=False),
            nn.BatchNorm2d(out_c),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_c, out_c, 3, padding=1, bias=False),
            nn.BatchNorm2d(out_c),
            nn.ReLU(inplace=True)
        )
    def forward(self, x): return self.conv(x)

class UNetResNet34(nn.Module):
    def __init__(self, n_classes=5, pretrained=True):
        super().__init__()
        resnet = models.resnet34(weights=models.ResNet34_Weights.DEFAULT if pretrained else
        for p in resnet.parameters(): p.requires_grad = False

        self.inc = nn.Sequential(resnet.conv1, resnet.bn1, resnet.relu)
        self.pool1 = resnet.maxpool
        self.encoder1 = resnet.layer1
        self.encoder2 = resnet.layer2
        self.encoder3 = resnet.layer3
        self.encoder4 = resnet.layer4

        self.up1 = nn.ConvTranspose2d(512, 256, 2, stride=2)
        self.conv1 = DoubleConv(512, 256)
        self.up2 = nn.ConvTranspose2d(256, 128, 2, stride=2)
        self.conv2 = DoubleConv(256, 128)
```

Model Evaluation

Objectives

- Compute mean Dice score on validation set

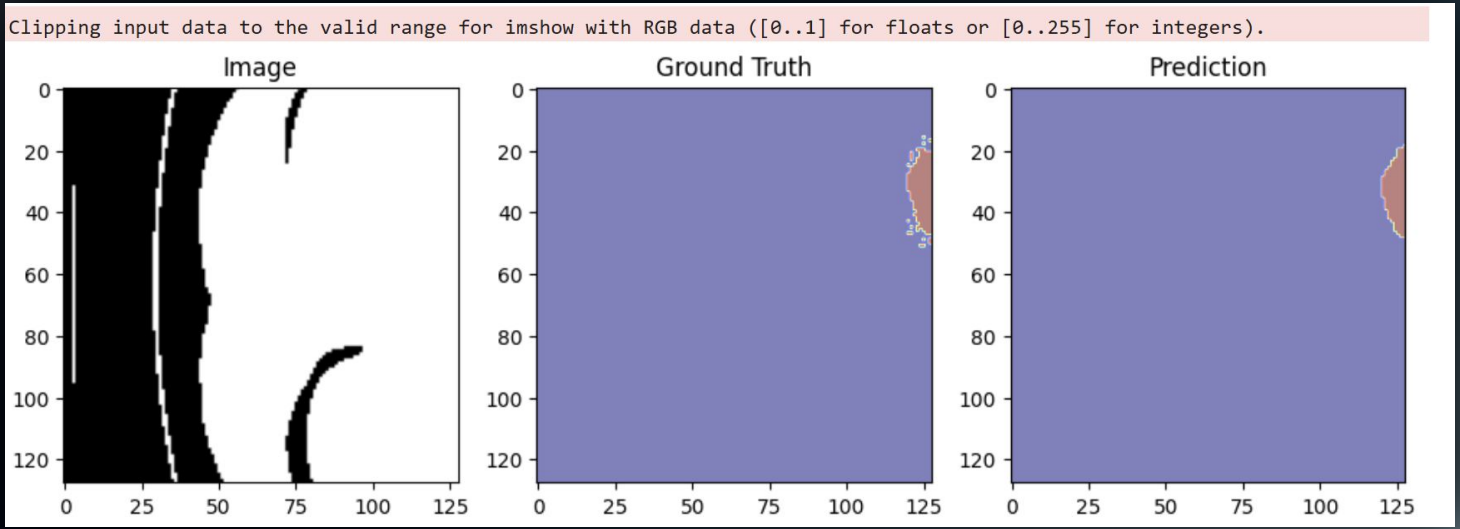
Key Highlights

- Dataset preloaded: 64,407 samples
- Validation samples evaluated: 16,102
- Average Dice (Stage 2): 0.9630

Visual Diagnostics

- Overlay RGB input, ground-truth, and predicted masks

Model Evaluation



Test Dataset & Single Image Prediction

Test Dataset

- UNet–ResNet34 model restored for inference
- TestDataset + DataLoader prepared (batch=8, shuffle=False)

Single Image Predictor

- predict_single(path) for on-demand segmentation
- Quick visual confirmation on arbitrary CT slices

Outcome

- Fast inference pipeline for both batch and single-image predictions

Challenges Faced

- Challenge: Inconsistent mask naming and missing files in datasets
  Solution: Regex-based mapping, skipped missing files, and logging.

- Challenge: Class imbalance across organs and tumors
  Solution: WeightedRandomSampler with dominant class caching.

- Challenge: Memory and GPU bottlenecks during training
  Solution: Patch-based extraction (128×128, stride 64), Preloading dataset optional, Mixed precision training (AMP)

- Challenge: Merging heterogeneous datasets (Pancreas + LITS)
  Solution: Unified labeling (0–4) and overlay visualization for verification.

- Challenge: Slow processing for patch extraction and saving
  Solution: Parallelized using ProcessPoolExecutor, avoiding lambda functions.

- Challenge: Model crashes due to torch.compile and datatype mismatch
  Solution: Disabled torch.compile, ensured img.float() and mask.long() types.

Conclusion

- Built a unified 5-class medical segmentation dataset (Pancreas + LiTS).
- Applied pixel-level balancing and patch extraction for training-ready inputs.
- Trained UNet–ResNet34 model with best Val Dice: 0.825.
- Performed quantitative and qualitative evaluation, including single-image predictions.
- Provides a reproducible end-to-end pipeline from dataset fusion to model inference.

<u>Significance</u>

- Accurate multi-organ and tumor segmentation aids clinical decision-making.
- Enables robust model training and benchmarking for research and healthcare applications.

Future Work

- Future Work (Expanded, Still Concise)
- 3D volumetric segmentation for richer spatial understanding.
- Cross-dataset validation to enhance generalizability across hospitals and scanners.
- Model compression & acceleration using ONNX/TensorRT for real-time deployment.
- Semi-supervised and self-supervised learning to leverage large unlabeled data.
- Active learning loop to iteratively improve the model with minimal annotations.
- Uncertainty estimation to highlight low-confidence regions for clinicians.
- Post-processing refinements such as CRFs or small-region filtering for cleaner masks.

References

- Ronneberger, O. et al. “U-Net: Convolutional Networks for Biomedical Image Segmentation,” MICCAI, 2015.
  https://arxiv.org/abs/1505.04597
- He, K. et al. “Deep Residual Learning for Image Recognition,” CVPR, 2016.
  https://arxiv.org/abs/1512.03385
- Çiçek, Ö. et al. “3D U-Net: Dense Volumetric Segmentation,” MICCAI, 2016.
  https://arxiv.org/abs/1606.06650
- Iglovikov, V. & Shvets, A. “TernausNet: U-Net Variants with VGG Pretraining,” 2018.
  https://arxiv.org/abs/1801.05746
- Albumentations Library. Fast image augmentation toolkit.
  https://albumentations.ai
- PyTorch Documentation. Model training, optimization, and inference.
  https://pytorch.org/docs/stable
- Medical Segmentation Decathlon. Dataset specification.
  http://medicaldecathlon.com
- Kingma, D. & Ba, J. “Adam Optimizer,” ICLR, 2015.
  https://arxiv.org/abs/1412.6980
- Sudre, C. et al. “Generalised Dice Loss,” DLMIA, 2017.
  https://arxiv.org/abs/1707.03237
- ResNet34 Pretrained Weights. Torchvision model zoo.
  https://pytorch.org/vision/stable/models/resnet.html

Thank You